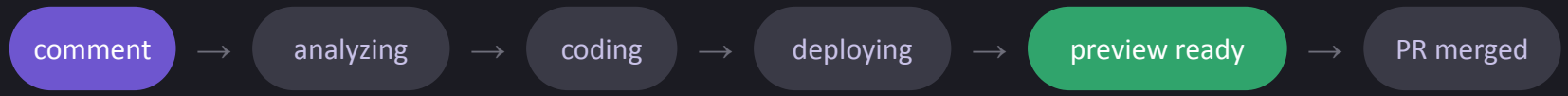
DESIGN RATIONALE — INTERVIEW POC

commentToFix

Comment on a live site like a Google Doc — an agent captures the runtime context, ships a fix to a preview deployment, and you iterate in the same thread until the PR merges.



Live demo **https://commenttofix.onrender.com** Repo **github.com/totorohxms/comment-to-fix**

Why this: feedback dies in transit

How UI feedback travels today

1. Designer spots a bug on staging, takes a screenshot
   Team dogfooding and share the bug to triage via slack
   Staging issue arises and we need to take screenshot to share
2. Screenshot lands in Slack, then becomes a ticket or we tag the agent to triage and address
3. Engineer tries to reproduce — people get tagged, build versions guessed, details chased
4. Fix ships later — and the context is lost on the product side too

The bet this POC makes

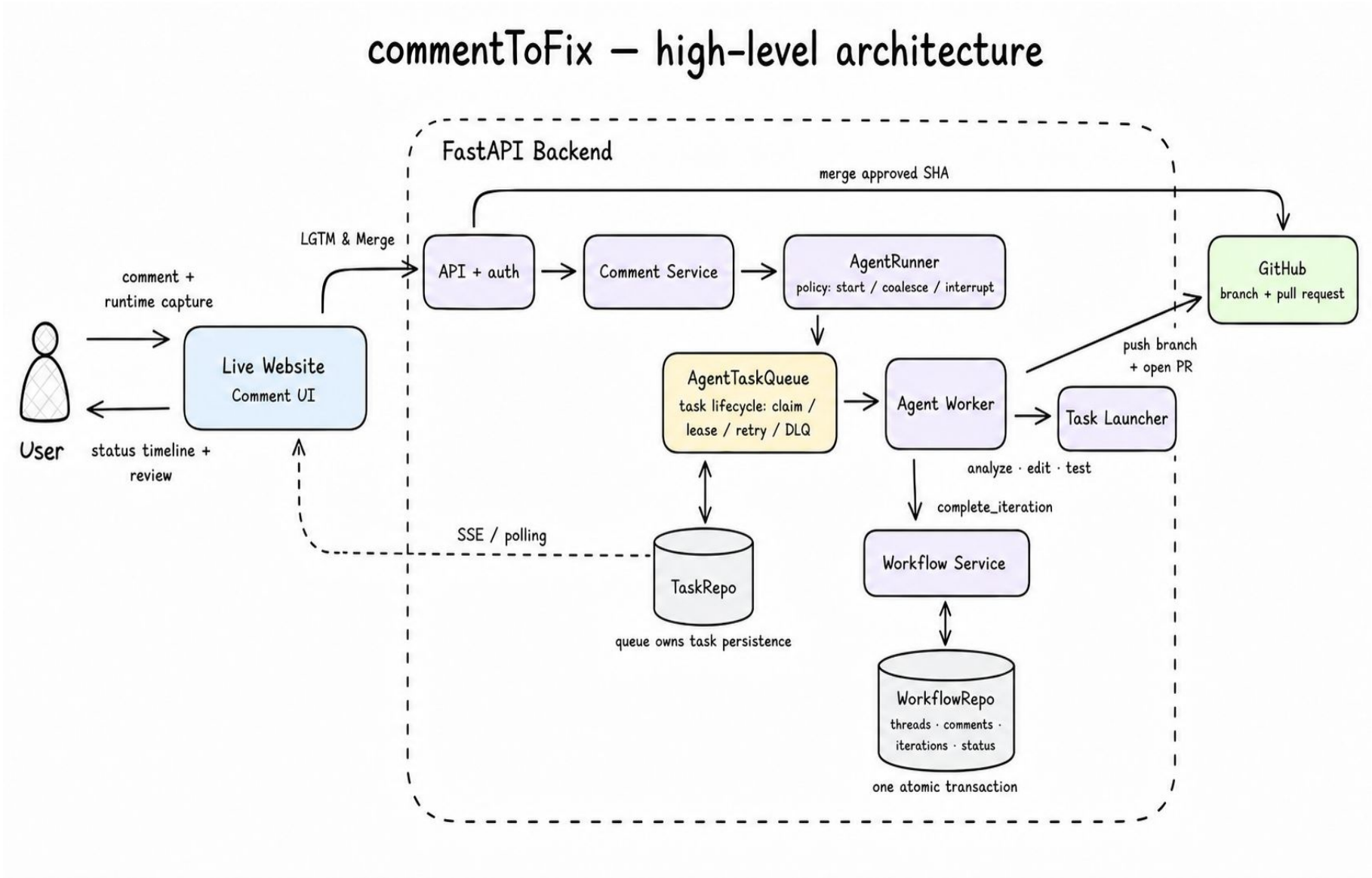
Comment where the bug lives — the product!
The product itself is the ticket: click an element, type, done.

Capture at click-time = perfect repro
Screenshot, DOM, network trace, console, sha, viewport — bundled the moment you comment. Click the capture chip on any comment to see exactly what the agent saw.

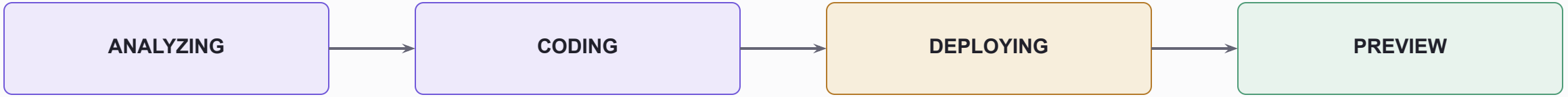
Iterate in the thread, not in Jira / Slack
The agent posts previews back into the conversation; follow-ups refine the same fix.

The idea: report, reproduce, fix, and verify — one conversation, right on the page.

Architecture layout



Comment Decision: make the thread a visible state machine



FOLLOW-UP RULES

STILL ANALYZING	**Fold it into the current ask**	Interpretation is not committed yet.
CODING HAS STARTED	**Queue the next iteration**	Keep the current diff tied to a stable request.
USER CANCELS	**Stop through the explicit cancel path**	Cancellation is a visible state change.

INVARIANT **Every decision is appended to the thread; every preview is tied to a SHA.**

Design philosophy: the thread is a document

Run the thread like a Doc: one living version, visible history, nothing changed in silence.

One living preview; shas are its version history

Each thread has one preview link that always shows the newest version — like a Doc URL. Every older version keeps its own frozen link. Open an old one and the thread reads as it was back then. Commenting there means: restart from here.

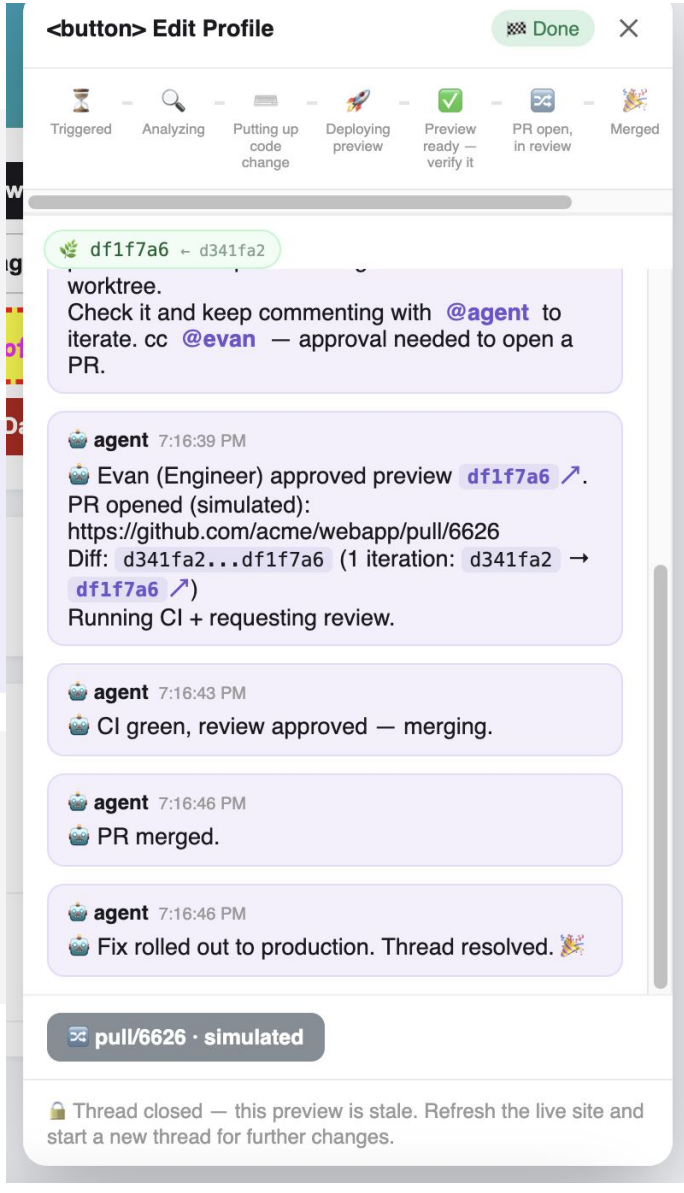
If the system does something, it says so

Behind the scenes, your comment can be combined with another, queued for the next round, or interrupt a running fix. That could feel random — so the agent posts what it did, in the thread: “combined your two asks”, “queued — starts after this deploy”, “this preview addresses X and Y”.

Asks have a commitment model

While the agent is still analyzing, you can change your ask. Once it starts coding, the ask is locked — wait for the preview or open a new thread. Cancel exists, and it's the only way to stop a run. Nothing changes silently.

Append-only all the way down: every decision the agent makes is a comment you can scroll back to — the conversation is the change record.



Key decisions & tradeoffs

Task granularity: one comment thread = one task, worktree, PR

The smallest reviewable unit, chosen on purpose. Bundling related comments can come later — carefully, as a product feature with batch limits and a clear owner.

\+ Targeted fixes: small diffs, easy review, one issue per PR, approver is always clear

− Overhead per fix — N comments mean N worktrees, N deploys, N PRs

→ *Only the runner knows what goes into a task, so revisiting this later stays cheap*

Comment arrival merges by thread lifecycle state

The thread's state decides what happens to your follow-up: still analyzing → folded in; already coding → it becomes the next fix. The thread tells you which happened.

\+ A fix never ships with half of what you asked for baked in

− Your follow-up sometimes waits for the next round instead of applying right away

→ *Predictability over immediacy — the state machine makes the rule visible, not magic*

Ownership: runner decides, queue delivers, agent executes

Three files, three jobs. runner.py decides (policy + narration), queue.py delivers (leases, retries, DLQ), agent.py executes. Each one is testable alone.

\+ Each concern has a clear owner, so failures and future changes stay localized.

− Three components create more conceptual overhead than a single loop.

→ *Features such as comment bundling change runner policy without affecting queue delivery or agent execution.*

Also decided: every fake sits behind a real interface (the swap is an env var; PRs already go real with a GitHub token) · SQLite + in-process queue — zero ops, one instance, chosen knowingly

Next Steps

Make it real

Real Claude Code launcher

A worktree per iteration off the base sha; capture bundle handed over as context; diff returned through the existing TaskLauncher seam.

Per-sha preview environments

PRs are already real: approve opens a branch + PR, merge squash-merges. Next: real per-sha deploys instead of in-app preview pages.

Real auth & sessions

Replace the user header with sessions/SSO; roles already modeled and enforced.

Make it safe & scalable

Capture privacy hardening

Allowlist what capture may record, expire old bundles, encrypt at rest, and only let thread participants open them.

Postgres + real broker

Lift the one-instance ceiling. The queue's semantics already match a real broker, so this is a port, not a redesign.

Embeddable widget SDK

One script tag adds commenting to any staging site. The demo site becomes just one host among many.

P0 product-market fit: pre-production, by design

Local — the engineer
docker compose up and the whole loop runs on your machine — interact with the e2e flow before anything is rolled out

→

Pre-prod / staging — the internal team
the P0 market: permissioned teammates comment on staging — safe data, real workflow, roles already enforced

→

Production / enterprise — later, on purpose
out of P0 scope until the hardening above lands: SSO, audit, capture privacy

Every extension lands behind an interface that already exists — nothing here requires re-architecting.

Scope & effort

~5 hrs

total build time

95

tests, running in ~2 s
on shrunk timers

2

services on Render
deployed from a Blueprint

Try it now

https://commenttofix.onrender.com
“@agent this button style is not right” …then watch the thread.

Song Feng · fengsong90@gmail.com · Aug 2026